SaveConfirmDialog UI仕様書

1. 概要

1.1 コンポーネントの目的

SaveConfirmDialogは、Petri Net編集画面でMAIMLファイルを保存する際に、保存先（S3またはローカルダウンロード）とバリデーション結果を確認するモーダルダイアログコンポーネントです。バリデーションエラーがある場合は、自動補正、強制保存、キャンセルのいずれかを選択できます。

主な機能:

- バリデーション結果の表示（エラー、警告、自動補正可能項目の統計）
- エラー詳細のプレビュー表示（上位3件）
- 保存先の選択（S3/ローカルダウンロード）
- S3フォルダの階層構造表示とツリー選択
- 新規フォルダパスの手動入力
- 自動補正保存/強制保存/通常保存/キャンセルの選択

1.2 使用場面

- **表示タイミング**:
  - Petri Net編集画面（PetriNetEditorInner）で保存処理を実行した時
  - SaveDialogで保存ボタンをクリックした後
  - MAIMLバリデーションが完了した後
- **表示条件**: ユーザーが保存を確定する前の最終確認
- **操作**: 保存先とアクションを選択して実行

1.3 アクセス権限

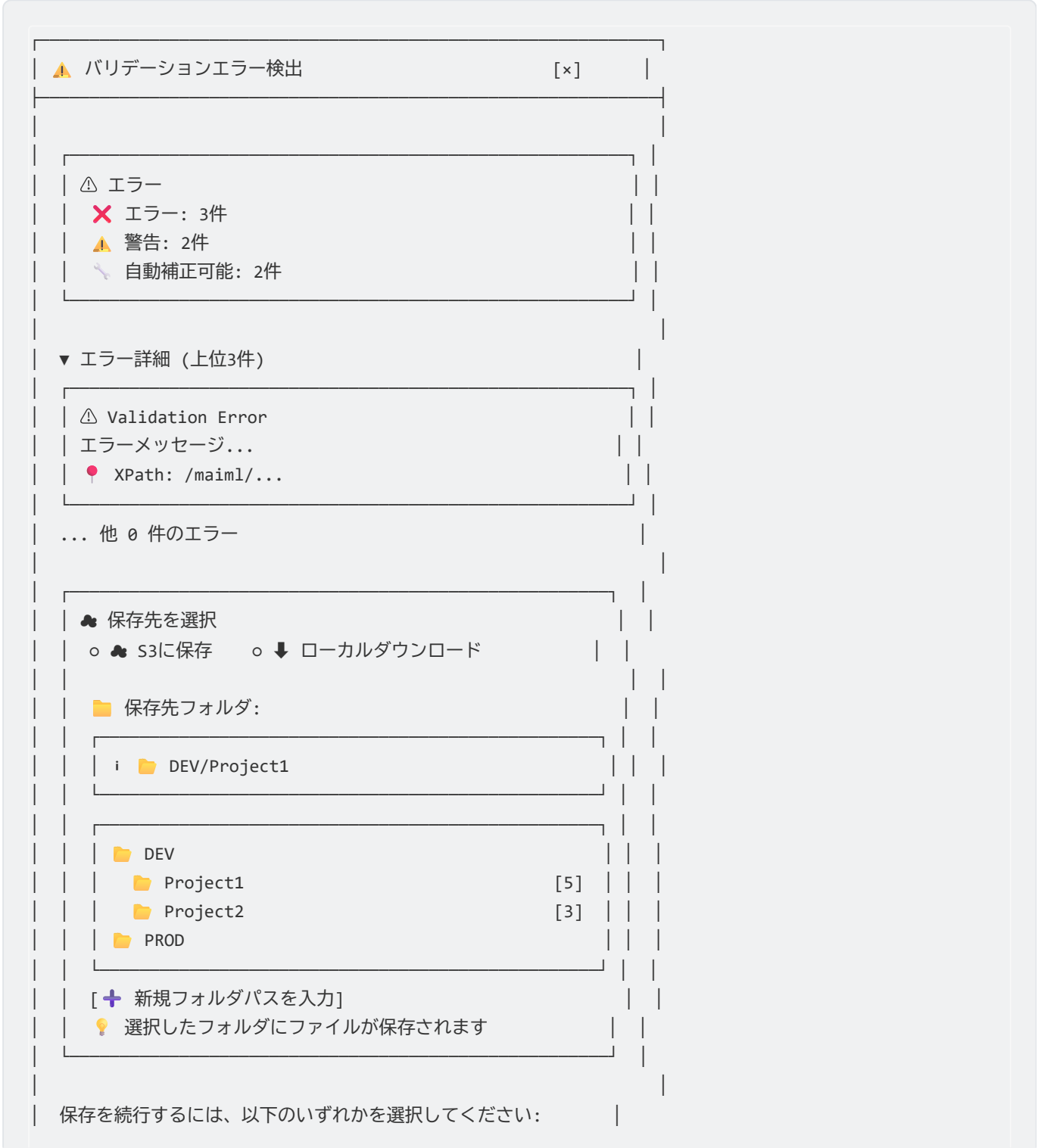
- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

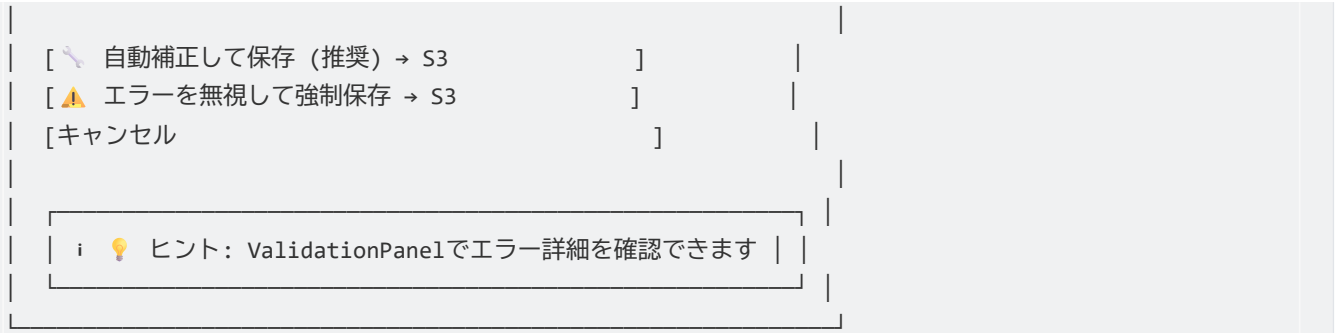
- **テナント分離**: organization_idによる分離（S3フォルダ取得時に適用）

---

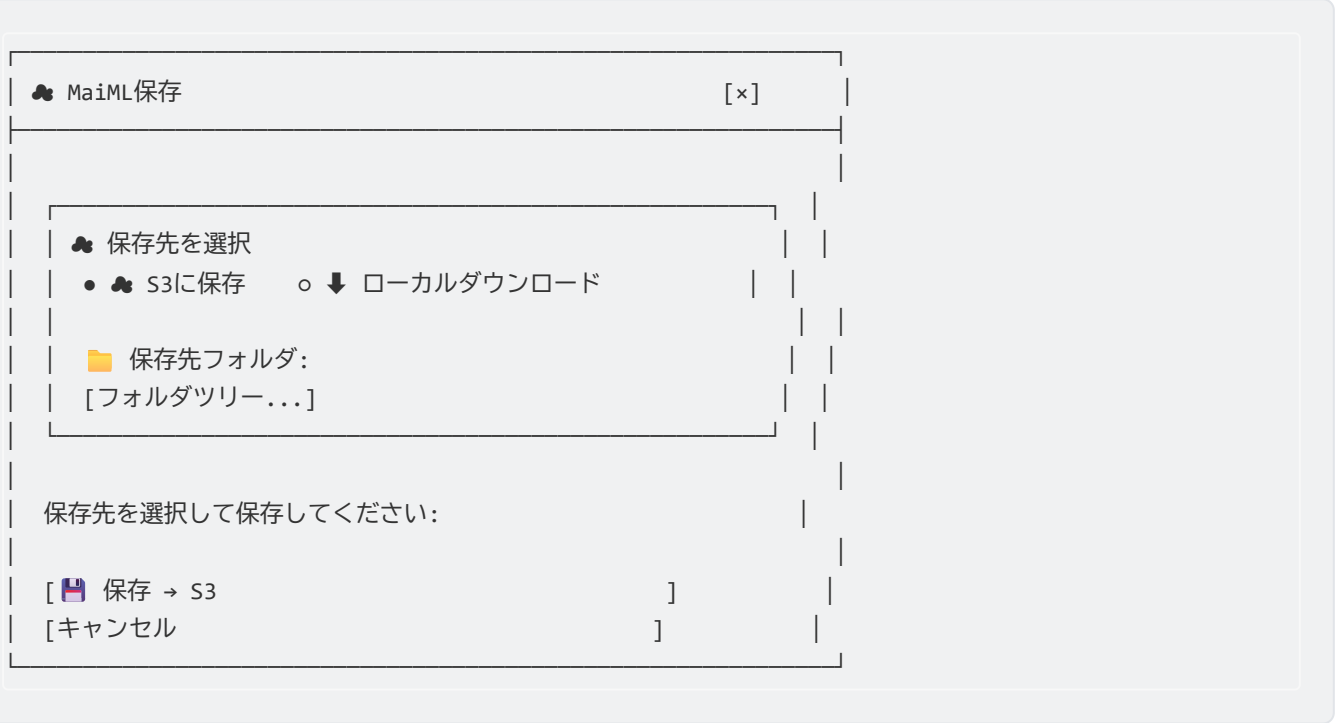
2. 画面構成

2.1 レイアウト構造（エラーあり）





2.2 レイアウト構造（エラーなし）



2.3 主要UI要素

2.3.1 モーダルヘッダー

- **タイトル（エラーあり）**: "⚠️ バリデーションエラー検出"
- **タイトル（エラーなし）**: "☁ MaiML保存"
- **アイコン**: WarningOutlined（エラーあり）/ CloudUploadOutlined（エラーなし）
- **色**: 赤（エラーあり）/ 通常（エラーなし）
- **幅**: 700px

2.3.2 バリデーション結果統計（エラーがある場合のみ）

- **表示条件**: `validationResult.errors.length > 0`
- **タイプ**: Alert（error）
- **内容**:
  - ❌ エラー: {errorCount}件
  - ⚠️ 警告: {warningCount}件
  - 🔧 自動補正可能: {correctionsCount}件

2.3.3 エラー詳細プレビュー（Collapse）

- **表示条件**: `validationResult.errors.length > 0`
- **ヘッダー**: "エラー詳細 (上位3件)"
- **内容**: 各エラーをAlert（error）で表示
  - タイトル: error.type
  - 説明: error.message または error.description
  - XPath: error.xpath（存在する場合）
- **表示件数**: 上位3件のみ
- **残件数表示**: "... 他 X 件のエラー"

2.3.4 保存先選択パネル

- **背景色**: #e6f4ff（水色）
- **枠線**: 2px solid #1677ff
- **角丸**: 8px
- **タイトル**: "☁ 保存先を選択"

Radio Group:

- **S3に保存**: ☁ CloudUploadOutlined
- **ローカルダウンロード**: ⬇ DownloadOutlined

2.3.5 S3フォルダ選択（S3選択時のみ）

選択中フォルダ表示:

- Alert（info）で現在選択中のフォルダを表示
- アイコン: FolderOpenOutlined
- テキスト: フォルダパス（例: "DEV/Project1"）

フォルダツリー:

- コンポーネント: Ant Design Tree
- 高さ: 最大300px（スクロール可能）
- アイコン: FolderOutlined
- ファイル数表示: Tag（フォルダ内のファイル数）
- 選択スタイル: 青色ハイライト
- 展開/折りたたみ: 可能（1階層目はデフォルトで展開）

新規フォルダパス入力:

- ボタン: "➕ 新規フォルダパスを入力"
- Input: プレースホルダー "例: DEV/MyProject/Data"
- 戻るボタン: "既存フォルダから選択"

2.3.6 アクションボタン

自動補正ボタン（エラーあり＆補正可能な場合のみ）:

- ラベル: "🔧 自動補正して保存 (推奨)"
- タイプ: primary
- 色: 緑（#52c41a）
- サイズ: large
- アイコン: ToolOutlined
- Tag: "→ S3" または "→ ダウンロード"

保存ボタン（エラーなし）/ 強制保存ボタン（エラーあり）:

- ラベル:
  - エラーなし: "💾 保存"

    - エラーあり: "⚠️ エラーを無視して強制保存"

- タイプ:
    - エラーなし: primary
    - エラーあり: default + danger
- アイコン:
    - エラーなし: CloudUploadOutlined
    - エラーあり: WarningOutlined
- Tag: "→ S3" または "→ ダウンロード"

キャンセルボタン:

- ラベル: "キャンセル"
- アイコン: CloseCircleOutlined
- タイプ: default

2.3.7 ヒント表示（エラーがある場合のみ）

- Alert（info）
- メッセージ: "💡 ヒント: ValidationPanelでエラー詳細を確認できます"

---

3. 機能仕様

3.1 バリデーション結果の表示

3.1.1 統計情報の計算

処理:

```
const errorCount = validationResult?.errors?.length || 0;
const warningCount = validationResult?.warnings?.length || 0;
const correctionsCount = validationResult?.corrections?.length || 0;
```

```
const hasErrors = errorCount > 0;
const hasCorrections = correctionsCount > 0;
```

UI動作:

- エラー数、警告数、自動補正可能数を集計
- エラーが1件以上ある場合、バリデーションエラー表示モードになる

3.1.2 エラー詳細のプレビュー

処理:

```
{validationResult.errors.slice(0, 3).map((error, index) => (
  <Alert
    key={index}
    message={error.type || 'Validation Error'}
    description={
      <>
        <Text>{error.message || error.description || 'エラー詳細なし'}</Text>
        {error.xpath && (
          <div>
            <Text type="secondary">📍 XPath: {error.xpath}</Text>
          </div>
        )}
      </>
    }
    type="error"
    showIcon
  />
))}
```

UI動作:

- 上位3件のエラーを表示
- 各エラーをAlertコンポーネントで表示
- XPathが存在する場合は場所を表示
- 残りのエラー件数を表示

3.2 保存先の選択

3.2.1 保存先切り替え

State:

```
const [saveDestination, setSaveDestination] = useState<'s3' | 'download'>('s3');
```

UI動作:

- デフォルトは "s3"
- Radio Groupで選択を切り替え
- "s3" 選択時: S3フォルダ選択UIを表示
- "download" 選択時: フォルダ選択UIを非表示

3.3 S3フォルダの取得と表示

3.3.1 フォルダー一覧の取得

処理フロー:

```
const fetchS3Folders = async () => {
  setLoadingFolders(true);

  try {
    // 1. LocalStorageからorganizationIdを取得
    const userInfoStr = localStorage.getItem('userInfo');
    const userInfo = userInfoStr ? JSON.parse(userInfoStr) : null;
    const currentOrgId = userInfo?.organizationId;

    // 2. API呼び出し
    const response = await fetch(
      `/api/s3/folders?organizationId=${currentOrgId}`,
      {
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
      }
    );

    const result = await response.json();

    // 3. フォルダリストをStateにセット
    if (result.success && result.folders) {
```

```
      setFolders(result.folders);
      buildTreeData(result.folders);

      // 4. デフォルトフォルダを設定
      if (sourceFolder) {
        setSelectedFolder(sourceFolder);
      } else if (result.folders.length > 0) {
        setSelectedFolder(result.folders[0].fullPath.replace(/\/$/, ''));
      }
    }
  } catch (error) {
    console.error('フォルダ取得エラー:', error);
    setFolders([]);
  } finally {
    setLoadingFolders(false);
  }
};
```

実行タイミング:

```
useEffect(() => {
  if (open && saveDestination === 's3') {
    fetchS3Folders();
  }
}, [open, saveDestination]);
```

UI動作:

1. ダイアログが開かれ、S3が選択されている場合にフォルダを取得
2. ローディング中はSpinを表示
3. 取得成功時、フォルダツリーを構築
4. デフォルトフォルダを選択（sourceFolder優先、なければ最初のフォルダ）

3.3.2 フォルダツリーの構築

処理フロー:

```
const buildTreeData = (folderList: S3Folder[]) => {
  const structure: FolderStructure = {};
  const defaultExpanded: React.Key[] = [];

  // 1. フォルダパスを階層構造に変換
```

```
  folderList.forEach(folder => {
    const parts = folder.fullPath.split('/').filter(p => p);
    let current = structure;

    for (let i = 0; i < parts.length; i++) {
      const part = parts[i];
      const currentPath = parts.slice(0, i + 1).join('/');

      if (!current[part]) {
        current[part] = {
          __isFolder: true,
          __folderName: part,
          __folderPath: currentPath,
          __depth: i,
          __fileCount: folder.fileCount || 0,
        };

        // 1階層目はデフォルトで展開
        if (i === 0) {
          defaultExpanded.push(currentPath);
        }
      }

      current = current[part] as FolderStructure;
    }
  });

  // 2. TreeノードのDataNode[]に変換
  const tree = buildTreeNodes(structure);
  setTreeData(tree);
  setExpandedKeys(defaultExpanded);
};
```

UI動作:

- フォルダパス（例: "DEV/Project1/Data"）を階層構造に変換
- Ant Design TreeのDataNode形式に変換
- 各ノードにフォルダアイコンとファイル数を表示
- 1階層目（ルートフォルダ）はデフォルトで展開

3.3.3 フォルダ選択

処理:

```
const handleFolderSelect = useCallback((selectedKeys: React.Key[]) => {
  if (selectedKeys.length > 0) {
    setSelectedFolder(selectedKeys[0] as string);
  }
}, []);
```

UI動作:

- ツリーのノードをクリックするとフォルダが選択される
- 選択されたフォルダパスが `selectedFolder` Stateにセットされる
- 選択中フォルダのAlertが更新される

3.3.4 新規フォルダパス入力

処理:

```
// 新規入力モードに切り替え
setShowCustomFolderInput(true);

// 既存フォルダ選択モードに戻す
setShowCustomFolderInput(false);
setCustomFolder('');
```

UI動作:

- "➕ 新規フォルダパスを入力" ボタンをクリック
- フォルダツリーが非表示になり、Inputフィールドが表示される
- カスタムパスを入力（例: "DEV/MyProject/Data"）
- "既存フォルダから選択" ボタンで元に戻る

3.3.5 ソースフォルダの自動設定

処理:

```
useEffect(() => {
  if (open && saveDestination === 's3' && sourceFolder && !selectedFolder) {
    console.log('✅ ソースフォルダを初期値に設定:', sourceFolder);
```

```
      setSelectedFolder(sourceFolder);
    }
  }, [open, saveDestination, sourceFolder, selectedFolder]);
```

UI動作:

- `sourceFolder` propsが存在する場合、それを初期選択フォルダに設定
- 元のファイルと同じフォルダに保存するための機能

3.4 保存アクションの実行

3.4.1 アクション選択

処理:

```
const handleChoice = (action: 'save' | 'auto-correct' | 'force-save' | 'cancel') => {
  const folder = showCustomFolderInput ? customFolder : selectedFolder;
  onSave(action, saveDestination, folder);
};
```

アクション種別:

アクション	説明	表示条件
save	通常保存	エラーなし
auto-correct	自動補正して保存	エラーあり＆補正可能
force-save	エラーを無視して強制保存	エラーあり
cancel	キャンセル	常に

UI動作:

1. ボタンクリック時に `handleChoice()` を呼び出し
2. 選択中のフォルダパスを取得（カスタム入力 or ツリー選択）
3. `onSave(action, destination, folder)` propsを呼び出し
4. 親コンポーネント（PetriNetEditorInner）が実際の保存処理を実行

---

4. データ仕様

4.1 Props定義

```
interface SaveConfirmDialogProps {
  open: boolean;                          // ダイアログの表示/非表示
  onClose: () => void;                    // 閉じる時のコールバック
  validationResult?: ValidationResult | null;  // バリデーション結果
  onSave: (action: 'save' | 'auto-correct' | 'force-save' | 'cancel', destination: string,
folder?: string) => void;  // 保存時のコールバック
  sourceFolder?: string;                  // ソースフォルダパス（初期値）
}
```

Props説明:

Prop名	型	必須	デフォルト値	説明
open	boolean	✅	-	ダイアログの表示状態
onClose	() => void	✅	-	ダイアログを閉じる時に呼ばれるコールバック
validationResult	ValidationResult | null	-	undefined	MAIMLバリデーション結果（エラー、警告、補正情報）
onSave	(action, destination, folder?) => void	✅	-	保存アクション実行時のコールバック
sourceFolder	string	-	""	ソースフォルダパス（デフォルト選択に使用）

4.2 ValidationResult型定義

```
interface ValidationError {
  type?: string;         // エラータイプ
  message?: string;      // エラーメッセージ
  description?: string;  // エラー説明
  xpath?: string;        // エラー発生箇所のXPath
}

interface ValidationResult {
  errors?: ValidationError[];    // エラー一覧
```

```
  warnings?: ValidationError[];  // 警告一覧
  corrections?: unknown[];       // 自動補正可能項目一覧
}
```

4.3 S3Folder型定義

```
interface S3Folder {
  fullPath: string;    // フォルダの完全パス（例: "DEV/Project1"）
  fileCount?: number;  // フォルダ内のファイル数
}
```

4.4 State定義

```
// 保存先選択
const [saveDestination, setSaveDestination] = useState<'s3' | 'download'>('s3');

// S3フォルダ関連
const [folders, setFolders] = useState<S3Folder[]>([]);
const [selectedFolder, setSelectedFolder] = useState<string>('');
const [customFolder, setCustomFolder] = useState<string>('');
const [loadingFolders, setLoadingFolders] = useState(false);
const [showCustomFolderInput, setShowCustomFolderInput] = useState(false);
const [treeData, setTreeData] = useState<DataNode[]>([]);
const [expandedKeys, setExpandedKeys] = useState<React.Key[]>([]);
```

State説明:

State名	型	説明
saveDestination	's3' | 'download'	保存先（S3またはダウンロード）
folders	S3Folder[]	S3フォルダー一覧（APIから取得）
selectedFolder	string	選択中のフォルダパス
customFolder	string	カスタム入力フォルダパス
loadingFolders	boolean	フォルダ読み込み中フラグ
showCustomFolderInput	boolean	カスタム入力モード表示フラグ
treeData	DataNode[]	Ant Design Tree用のデータ

expandedKeys	React.Key[]	展開中のツリーノードのキー

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
1	S3フォルダー一覧取得	GET	`/api/s3/folders?organizationId={orgId}`	✅ 実装済み	組織のS3フォルダー一覧を取得

5.2 API呼び出しコード

5.2.1 S3フォルダー一覧取得

リクエスト:

```
const response = await fetch(
  `/api/s3/folders?organizationId=${currentOrgId}`,
  {
    headers: {
      'Content-Type': 'application/json',
    },
    credentials: 'include',
  }
);
```

レスポンス (200 OK):

```
{
  "success": true,
  "folders": [
    {
      "fullPath": "DEV/Project1",
      "fileCount": 5
    },
    {
```

```
      "fullPath": "DEV/Project2",
      "fileCount": 3
    },
    {
      "fullPath": "PROD/Production1",
      "fileCount": 10
    }
  ]
}
```

5.3 データフロー

```
SaveConfirmDialog
  ↓ ダイアログ表示時（S3選択時）
fetchS3Folders()
  ↓ LocalStorageからorganizationId取得
GET /api/s3/folders?organizationId={orgId}
  ↓ 成功
フォルダー一覧を取得
  ↓
buildTreeData() - ツリー構造に変換
  ↓
Tree表示 + デフォルトフォルダ選択

SaveConfirmDialog
  ↓ ボタンクリック
handleChoice(action)
  ↓
onSave(action, destination, folder)
  ↓
PetriNetEditorInner（親コンポーネント）
  ↓ 実際の保存処理を実行
POST /api/v1/experiment-admin/maiml/save-to-s3
  または
ローカルダウンロード処理
```

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	発生タイミング	対処方法
フォルダ取得失敗	API呼び出しエラー	console.errorで記録、空の配列を設定
organizationId未取得	LocalStorageに存在しない	console.errorで記録、処理を中断
フォルダが存在しない	API結果が空	"フォルダが見つかりません" Alertを表示
カスタムフォルダパスが空	新規入力モードで空文字	親コンポーネントで検証（SaveConfirmDialogでは検証なし）

6.2 エラー処理コード

6.2.1 フォルダ取得エラー

```
try {
  const response = await fetch(`/api/s3/folders?organizationId=${currentOrgId}`, ...);
  const result = await response.json();

  if (result.success && result.folders) {
    setFolders(result.folders);
    buildTreeData(result.folders);
  }
} catch (error) {
  console.error('❌ [SaveConfirmDialog] フォルダ取得エラー:', error);
  setFolders([]);
} finally {
  setLoadingFolders(false);
}
```

6.2.2 organizationId未取得

```
const userInfoStr = localStorage.getItem('userInfo');
const userInfo = userInfoStr ? JSON.parse(userInfoStr) : null;
const currentOrgId = userInfo?.organizationId;

if (!currentOrgId) {
  console.error('❌ [SaveConfirmDialog] Organization ID not found');
  setFolders([]);
  setLoadingFolders(false);
```

```
  return;
}
```

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/SaveConfirmDialog.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/editorv2/PetriNetEditorInner.tsx` - SaveConfirmDialogを呼び出す
- **API**: `/api/s3/folders` - S3フォルダ一覧取得API

7.3 依存コンポーネント

Ant Designコンポーネント:

- `Modal` - モーダルダイアログ
- `Radio`, `Radio.Group` - 保存先選択
- `Input` - カスタムフォルダパス入力
- `Button` - アクションボタン
- `Alert` - エラー表示、情報表示
- `Tree` - フォルダツリー表示
- `Collapse`, `Panel` - エラー詳細の折りたたみ表示
- `Tag` - ファイル数表示、保存先表示
- `Space` - レイアウト
- `Typography` (Text, Title) - テキスト表示

- `Spin` - ローディングインジケータ

Ant Designアイコン:

- `FolderOutlined` - フォルダアイコン
- `FolderOpenOutlined` - 選択中フォルダアイコン
- `CloudUploadOutlined` - S3保存アイコン
- `DownloadOutlined` - ダウンロードアイコン
- `WarningOutlined` - 警告アイコン
- `ToolOutlined` - 自動補正アイコン
- `CloseCircleOutlined` - キャンセルアイコン

7.4 使用例

```
// PetriNetEditorInner.tsx
const [saveConfirmOpen, setSaveConfirmOpen] = useState(false);
const [validationResult, setValidationResult] = useState<ValidationResult | null>(null);

const handleSaveConfirm = (
  action: 'save' | 'auto-correct' | 'force-save' | 'cancel',
  destination: string,
  folder?: string
) => {
  if (action === 'cancel') {
    setSaveConfirmOpen(false);
    return;
  }

  // 保存処理を実行
  if (destination === 's3') {
    saveToS3(action, folder);
  } else {
    downloadLocally(action);
  }
};

return (
  <>
    <SaveConfirmDialog
      open={saveConfirmOpen}
      onClose={() => setSaveConfirmOpen(false)}
```

```
      validationResult={validationResult}
      onSave={handleSaveConfirm}
      sourceFolder="DEV/Project1"
    />
  </>
);
```

---

8. 補足情報

8.1 バリデーションエラー表示のロジック

エラーありの判定:

```
const hasErrors = (validationResult?.errors?.length || 0) > 0;
```

表示切り替え:

- `hasErrors === true` : エラーモード（赤いヘッダー、自動補正/強制保存ボタン表示）
- `hasErrors === false` : 通常モード（青いヘッダー、通常保存ボタン表示）

8.2 フォルダツリーの階層構造

入力データ:

```
[
  { fullPath: "DEV/Project1", fileCount: 5 },
  { fullPath: "DEV/Project2", fileCount: 3 },
  { fullPath: "PROD/Production1", fileCount: 10 },
]
```

変換後のツリー構造:

```
📁 DEV
  📁 Project1 [5]
  📁 Project2 [3]
```

```
📁 PROD
  📁 Production1 [10]
```

8.3 マルチテナント分離

organizationIdの取得:

- LocalStorageの `userInfo` から取得
- API呼び出し時にクエリパラメータとして送信
- バックエンドでorganization_idによるフィルタリングを実施

セキュリティ:

- 他の組織のフォルダは取得・表示されない
- APIレベルでテナント分離を保証

8.4 保存先タグの表示

```
<Tag style={{ marginLeft: 8 }}>
  → {saveDestination === 's3' ? 'S3' : 'ダウンロード'}
</Tag>
```

目的:

- ユーザーが選択した保存先を視覚的に確認できる
- ボタンクリック前に保存先を再確認できる

8.5 自動補正機能

自動補正ボタンの表示条件:

```
{hasErrors && hasCorrections && (
  <Button
    type="primary"
    icon={<ToolOutlined />}
    onClick={() => handleChoice('auto-correct')}
    style={{ backgroundColor: '#52c41a' }}
```

```
  >
    🔧 自動補正して保存（推奨）
  </Button>
)}
```

表示条件:

- エラーがある（`hasErrors === true`）
- 自動補正可能な項目がある（`hasCorrections === true`）

UI動作:

- 緑色のボタンで目立つように表示
- "(推奨)" ラベルで推奨アクションであることを示す

8.6 強制保存の注意喚起

強制保存ボタンの表示:

```
<Button
  type="default"
  danger={hasErrors}
  icon={<WarningOutlined />}
  onClick={() => handleChoice('force-save')}
>
  ⚠️ エラーを無視して強制保存
</Button>
```

UI設計:

- `danger` プロパティで赤色表示
- 警告アイコン（⚠️）で危険性を示す
- ラベルで "エラーを無視" と明示

目的:

- ユーザーに強制保存のリスクを認識させる
- 誤操作を防ぐ

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースでUI仕様書を作成	Cline
2026-01-29	V2保存確認ダイアログ実装（v2.0.0）	開発チーム